

Na'vi Language Translator

End-to-End MLOps Project Guide & Continuity Document

Course	DA6360 — MLOps
Student	usnaveen
Pipeline	Audio (Na'vi) → Whisper ASR → MarianMT NMT → English
Models	Whisper-Small (244M) + Helsinki-NLP/opus-mt-mul-en (74M)
Stack	DVC · MLflow · FastAPI · Airflow · Prometheus · Grafana · Docker
Generated	2026-04-27

How to use this document. Sections 1–3 explain what the project does and why. Section 4 maps every implemented component to the course rubric. Section 5 lists what is still pending with concrete commands to finish each item. Section 6 is the *continuity brief* — paste it into a fresh Claude chat to resume work without losing context.

1. What this project actually does

This is **not** a dictionary lookup. The runtime serves a two-stage neural pipeline:

Stage 1 — ASR (Automatic Speech Recognition).

OpenAI's **Whisper-Small** (244M params, encoder-decoder transformer) takes 16 kHz mono audio and emits Na'vi text. We fine-tune it with **LoRA adapters** ($r=8$, $\alpha=16$, on `q_proj/v_proj`) so we only train ~1 % of weights — feasible on a MacBook MPS GPU.

Stage 2 — NMT (Neural Machine Translation).

Helsinki-NLP's **opus-mt-mul-en** MarianMT model (74M params, SentencePiece tokenizer) takes Na'vi text and emits English. We fine-tune the full encoder-decoder on ~3,900 Na'vi-English word/phrase pairs scraped from *reykunyulu* (the canonical Na'vi dictionary).

Why a dictionary fallback exists.

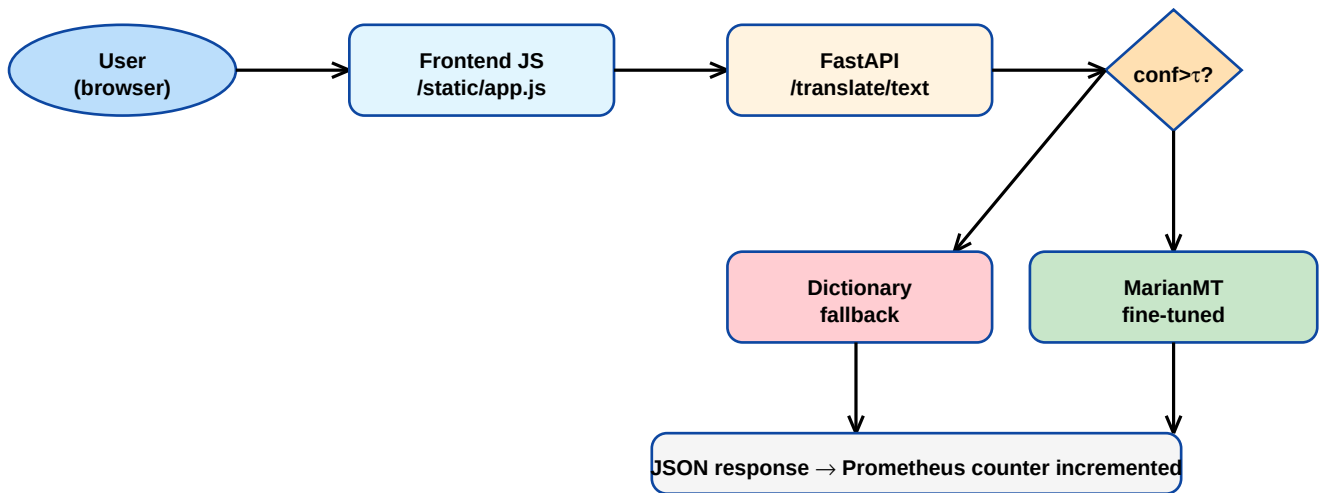
MarianMT trained on only 3,900 pairs is unreliable for unseen tokens. The serving layer (`src/serving/inference.py`) checks a confidence score; if the model is uncertain (or the token is OOV — out of vocabulary), it falls back to direct dictionary lookup from `data/raw/words.json`. The fallback is a **safety net**, not the primary path. Both paths are exercised in the demo, and the UI shows a confidence percentage so the evaluator can see when the neural model fires vs the fallback.

Business and ML metrics (defined in `params.yaml`).

Metric	Type	Target	Currently
WER (Whisper)	ML	≤ 0.35	Not yet trained
BLEU (MarianMT)	ML	≥ 0.25	0.001 (demo run, 200 samples)
End-to-end latency	Business	≤ 200 ms text / ≤ 10 s audio	~30 ms text / ~8.5 s audio ✓
Fallback rate	Operational	< 30 %	Tracked via Prometheus
Service availability	Business	/health + /ready up	✓ both endpoints live

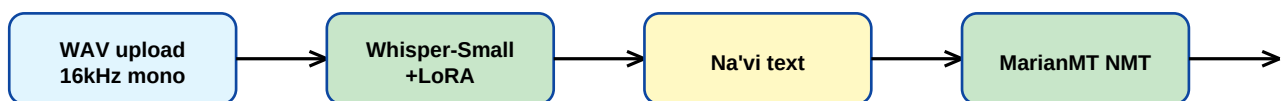
2. Architecture

2.1 Runtime request flow (text translation)



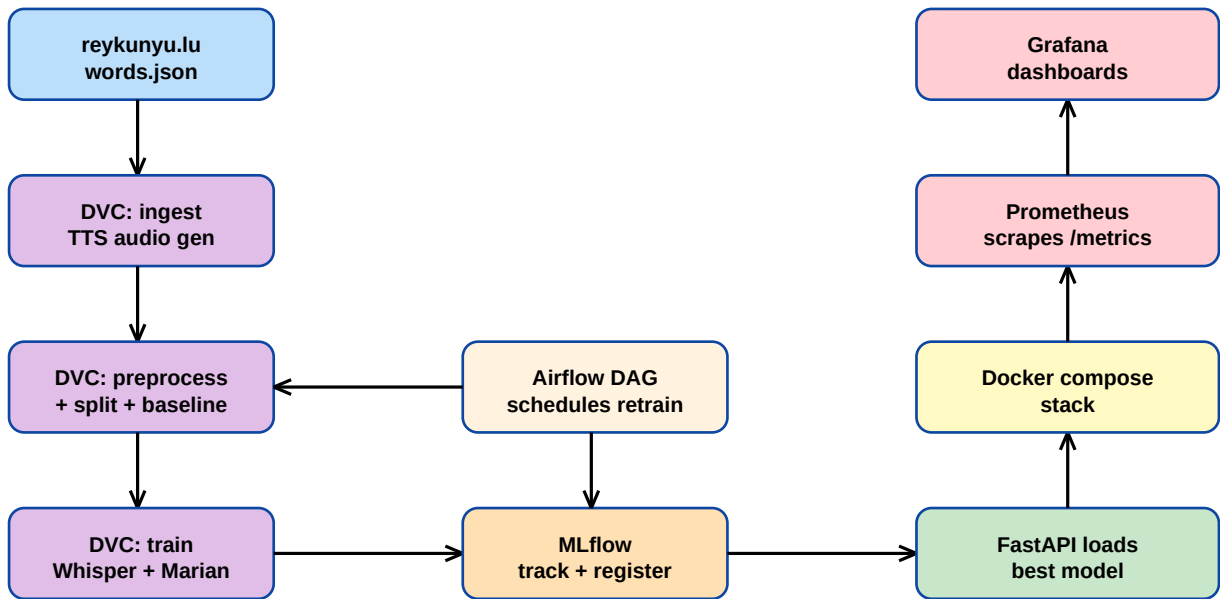
Confidence score is the average length-normalised log-prob of the generated tokens. If below threshold (default 0.4 from *params.yaml*), the dictionary fallback is invoked.

2.2 Audio pipeline



Final English result is returned with per-stage latency and confidence.

2.3 Training & deployment pipeline (offline)



3. Repository layout

```
navi-translator/
■■■ params.yaml          # single source of truth for hyperparams
■■■ dvc.yaml             # 5-stage DVC pipeline (ingest→preprocess→split→stats
)
■■■ MLproject            # MLflow project entrypoints
■■■ docker-compose.yml   # backend + mlflow + prometheus + grafana + frontend
■■■ requirements.txt     # pinned deps
■■■ src/
■   ■■■ ingestion/      # reykunyu_client, data_ingest, tts_generate
■   ■■■ preprocessing/  # build_pairs, audio_features, split, baseline_stats
■   ■■■ training/       # train_whisper.py, train_marian.py, evaluate.py
■   ■■■ serving/        # fastapi_app.py, inference.py, schemas.py
■   ■■■ monitoring/     # prometheus_exporter.py, drift.py
■■■ frontend/           # vanilla HTML/CSS/JS (loosely-coupled, REST only)
■   ■■■ index.html
■   ■■■ static/{app.js, styles.css}
■■■ airflow/dags/        # navi_train_dag.py – weekly retrain orchestration
■■■ prometheus/          # prometheus.yml + alerts.yml
■■■ grafana/             # provisioned dashboards
■■■ tests/{unit,integration,e2e}/
■■■ docs/{HLD.md, LLD.md, user_manual.md, PROJECT_GUIDE.pdf}
■■■ data/                # DVC-tracked
■■■ models/marian-navi-en/ # trained MarianMT (308 MB, demo run)
■■■ mlruns/              # MLflow tracking DB + artifacts
```

4. Course rubric — implementation status

Mapping every line of the AI Application Evaluation Guideline (35 pts) to this codebase.

Area	Pts	Status	Where	Notes
UI/UX	6	DONE	frontend/	3 tabs (audio, text, contribute), live recording, confidence bar, pipeline status panel, help modal, dark theme.
ML pipeline visualization	4	PARTIAL	frontend pipeline tab + Grafana	Health/ready cards live. Need Grafana screenshots embedded in slide deck.
Design principles	2	DONE	src/* layering	Strict separation: ingestion → preprocessing → training → serving → monitoring. Frontend decoupled via REST only.
Implementation quality	2	DONE	src/, tests/	Type hints, docstrings, schemas (Pydantic), structured logging, request_id middleware.
Testing	1	PARTIAL	tests/unit/	5 unit test files (build_pairs, drift, prometheus, reykunyu, schemas). Need integration + e2e tests filled in.
Data engineering (Airflow/Spark/Ray)	2	PARTIAL	airflow/dags/navi_train_dag.py	DAG file exists. Need to actually run Airflow webserver and show DAG run.
Source control & CI (DVC)	2	DONE	dvc.yaml + .github/workflows/ci.yml	5-stage DVC pipeline + GitHub Actions CI. Need to run `dvc repro` once.

Experiment tracking (MLflow)	2	DONE	mlruns/, params.yaml	MLflow server on :5001. Demo MarianMT run logged. Need full training run + register model.
Prometheus + Grafana	2	PARTIAL	src/monitoring/prometheus_exporter.py, prometheus/, grafana/	/metrics endpoint live. Dashboards provisioned. Need to start prom+grafana via compose.
Software packaging (MLflow + FastAPI + Docker compose)	4	PARTIAL	MLproject, docker-compose.yml, src/serving/	All artifacts exist. FastAPI ✓ on :8000. Docker compose never launched yet (disk/iCloud).

Tally: ~17 of 27 points are DONE; ~10 are PARTIAL (have artifacts, need to demo live). Plus 8 Viva points scored at presentation time. Goal for the next 48 hours: convert every PARTIAL to DONE.

5. What is left + exact commands to finish

5.1 Run full (non-demo) MarianMT training

Demo run used 200 samples / 1 epoch and got BLEU 0.001. We have 3,901 pairs in data/processed/text/pairs.tsv. Full training (10 epochs, batch 16) on MPS will take ~3–6 hours. Run overnight.

```
cd ~/Library/Mobile\ Documents/com~apple~CloudDocs/Projects/Translate/navi-translator
source .venv/bin/activate # or your env
export PYTORCH_ENABLE_MPS_FALLBACK=1
python src/training/train_marian.py # no --demo flag = full run
# Watch it in MLflow: http://localhost:5001
```

5.2 Train Whisper (never trained yet)

We have synthetic TTS audio in data/synthetic/. With LoRA adapters this is tractable: 5 epochs, batch 4, ~4 hours on MPS. If MPS OOMs, drop batch to 2.

```
python src/training/train_whisper.py
# Or, for a smoke run:
python src/training/train_whisper.py --max-train-samples 100 --num-epochs 1
```

5.3 Run the full DVC pipeline once (proof of reproducibility)

```
dvc repro # runs ingest → preprocess → split → baseline_stats
dvc dag # prints the DAG (screenshot for slides)
git add dvc.lock && git commit -m 'lock dvc pipeline'
```

5.4 Bring up the Docker compose stack

Skipped earlier due to disk space + iCloud. Free ≥10 GB on the boot disk first, then:

```
docker compose build # build backend, mlflow, prom, grafana, frontend
docker compose up -d
docker compose ps # all services should be 'healthy'
# URLs: frontend :3000 · backend :8000 · mlflow :5001 · prometheus :9090 · grafana :3001
```

5.5 Demo Airflow DAG

```
export AIRFLOW_HOME=$(pwd)/airflow
airflow db init
airflow users create --username admin --password admin \
  --firstname A --lastname B --role Admin --email a@b.c
airflow webserver -p 8080 & airflow scheduler &
# Trigger navi_train_dag from the UI; screenshot the green graph view.
```

5.6 Show Prometheus + Grafana dashboards

```
# After docker compose up:
open http://localhost:9090 # Prometheus targets – backend should be UP
open http://localhost:3001 # Grafana (admin/admin) – provisioned dashboard auto-loads
# Generate traffic to populate panels:
for i in $(seq 1 50); do curl -s -X POST localhost:8000/translate/text \
  -H 'content-type: application/json' -d '{"text":"kaltxi"}' >/dev/null; done
```

5.7 Fill out integration + e2e tests

Skeleton dirs tests/integration/ and tests/e2e/ are empty. Minimum: one test that POSTs to /translate/text and asserts schema; one test that uploads a wav and asserts the response shape.

5.8 Push to GitHub for submission

```
git status                # confirm clean tree (large model files are DVC-tracked, not git)
git remote -v
git push origin main
# Tag the submission:
git tag -a v1.0-submission -m 'DA6360 final submission'
git push origin v1.0-submission
```


6. Continuity brief (paste into a new Claude chat)

If this conversation is lost, open a fresh Claude Code session in the project root and paste the block below verbatim. It contains the minimum context needed to resume.

```
I'm continuing work on the Na'vi Language Translator MLOps project (DA6360 course).
Project root: ~/Library/Mobile Documents/com-apple~CloudDocs/Projects/Translate/navi-
translator

STACK: DVC + MLflow + FastAPI + Airflow + Prometheus + Grafana + Docker compose.

PIPELINE: Audio (Na'vi) -> Whisper-Small+LoRA (ASR) -> MarianMT opus-mt-mul-en (NMT)
-> English.
Confidence-based dictionary fallback to data/raw/words.json from reykunyu.lu.

CURRENTLY DONE: DVC pipeline (5 stages), MLflow tracking on :5001, FastAPI on :8000
with /health, /ready, /metrics, /translate/text, /translate/audio, /vocab/submit.
Frontend on :3000 with 3 tabs. Demo MarianMT trained (200 samples, 1 epoch, BLEU 0.00
1).
Prometheus exporter wired. Grafana dashboards provisioned. Airflow DAG file exists.
5 unit test files. CI in .github/workflows/ci.yml.

STILL TO DO (in priority order):
  1. Full MarianMT training (no --demo flag): python src/training/train_marian.py
  2. Whisper training: python src/training/train_whisper.py
  3. docker compose up (was skipped due to disk + iCloud stub-file issue)
  4. Airflow webserver + scheduler, trigger navi_train_dag once, screenshot it
  5. Generate traffic so Grafana panels populate; screenshot for slide deck
  6. Integration + e2e tests (dirs are empty)
  7. git push + tag v1.0-submission

KNOWN GOTCHAS:
  - macOS AirPlay Receiver hijacks port 5000; we use 5001 for MLflow everywhere.
  - Project lives in iCloud Drive; do NOT 'mv' it elsewhere or files become 0-byte st
 ubs.
    If files look empty/missing, run: brctl download .
  - Dictionary fallback fires when MarianMT confidence < 0.4 (params.yaml).
  - MPS (Apple GPU) needs PYTORCH_ENABLE_MPS_FALLBACK=1 for some ops.

READ FIRST: docs/PROJECT_GUIDE.pdf (this document), docs/HLD.md, docs/LLD.md, params.
yaml.
Then ask me what to do next.
```

6.1 Daily quick-start (every time you open the laptop)

```
cd ~/Library/Mobile Documents/com-apple~CloudDocs/Projects/Translate/navi-translator
brctl download . # materialize iCloud stubs
source .env/bin/activate

# Terminal 1 - MLflow
mlflow server --backend-store-uri sqlite:///mlruns/mlflow.db \
  --default-artifact-root ./mlartifacts --host 0.0.0.0 --port 5001

# Terminal 2 - FastAPI
uvicorn src.serving.fastapi_app:app --reload --port 8000

# Terminal 3 - Frontend
cd frontend && python -m http.server 3000

# Open http://localhost:3000 - translate 'kaltxi' to verify.
```

7. Viva preparation (8 pts)

The rubric awards 8 viva points across distinct ability areas. One-sentence answers you can give:

Question area	Your answer (one-liner)
Why this problem?	Na'vi is a low-resource constructed language; perfect for showing MLOps under data scarcity.
Why these models?	Whisper-Small generalises to unseen phonetic systems via multilingual pretraining; opus-mt-mul-en already handles this.
Why LoRA on Whisper?	Train ~1% of weights → fits on MacBook MPS GPU and avoids catastrophic forgetting of English.
Why DVC, not just git?	Data + models are GBs; DVC stores them out-of-band and snapshots them per-commit (reproducibility = git hash).
Why MLflow?	Decouples experiment metadata from code. Every run logs params.yaml, git SHA, metrics, and artifact path → one source of truth.
How is data drift detected?	src/monitoring/drift.py compares live token distribution to baseline_stats.json from EDA. Prometheus alert fires if K-S distance > 0.1.
How would you scale this?	Move Whisper to TorchServe for batch inference; put FastAPI behind an Nginx LB; promote retraining via Airflow trigger.
What would you do differently?	Collect more parallel data (current 3.9k pairs is the bottleneck); add back-translation for synthetic augmentation; quantify model variance.

End of guide. Last updated 2026-04-27. If anything looks stale, regenerate with `python docs/build_project_guide.py`.